

# Sequências Female e Male de Hofstadter

Trabalho 01 - Organização e Arquitetura de Processadores

Desenvolvedores: Juarez Fernando Goncalves dos Santos, Theo Carvalho Kirsch e Allan Rosa.

Data: 28/09/2026.

## Objetivo e definição

O programa implementa as sequências mutuamente recursivas de Hofstadter em Assembly MIPS, conforme o enunciado “Trabalho - Assembly do Processador MIPS - Hofstadter\_v2.pdf”, seções 1.1, 1.2 e 2.

-  $F(0) = 1$ ; para  $n > 0$ ,  $F(n) = n - M(F(n - 1))$ .

-  $M(0) = 0$ ; para  $n > 0$ ,  $M(n) = n - F(M(n - 1))$ .

Após o cabeçalho com data e desenvolvedores, main solicita um inteiro n. Se n for negativo, encerra. Caso contrário, imprime três linhas: os índices de 0 até n, seus valores Female e seus valores Male. Depois solicita outra entrada. O algoritmo em português estruturado está incluído neste PDF.

## Organização do Assembly

main controla a entrada e os três laços de impressão. \$s0 guarda o limite n e \$s1 guarda o índice da coluna. As funções female e male recebem o parâmetro em \$a0 e devolvem o resultado em \$v0. Elas não alteram \$s0 ou \$s1. \$t0 é temporário; não guarda um valor que precise sobreviver a outra chamada recursiva.

As três macros são print\_string (syscall 4), print\_int (syscall 1) e read\_int (syscall 5). O encerramento usa o syscall 10. As macros de saída utilizam \$v0 e \$a0; por isso, main copia o resultado da função para \$t0 antes de chamar print\_int.

Female primeiro calcula  $F(n - 1)$ , chama Male com esse resultado e subtrai o retorno do n original. Male faz o processo correspondente:  $M(n - 1)$ , Female desse resultado e a subtração. Assim, ambas têm chamadas próprias e cruzadas, preservando a recursividade exigida.

## Pilha e retorno

Cada chamada com  $n > 0$  decrementa \$sp em 8 bytes. O quadro é:

| Deslocamento em relação a \$sp | Conteúdo                 |
|--------------------------------|--------------------------|
| 0                              | Parâmetro n original     |
| 4                              | Endereço de retorno \$ra |

Depois das chamadas, n é carregado em \$t0 para a subtração. O endereço de retorno é restaurado, \$sp é incrementado em 8 e jr \$ra retorna ao chamador. O valor final de \$a0 não precisa ser preservado para o chamador. Os casos  $n = 0$  não fazem chamadas e retornam diretamente, sem reservar um quadro.

Em pilha-female.png, a execução de F(4) está dentro de F(3), antes de chamar F(2). Há dois quadros, e \$sp = 0x7ffefec, 16 bytes abaixo do valor inicial.

Em pilha-male.png, a cadeia ativa é F(4) → F(3) → F(2) → F(1) → M(1). A próxima chamada será M(0). Os cinco quadros ocupam 40 bytes:

| Função ativa | Endereço do quadro | n salvo | \$ra salvo |
|--------------|--------------------|---------|------------|
| M(1)         | 0x7ffefd4          | 1       | 0x00400174 |
| F(1)         | 0x7ffefdc          | 1       | 0x0040016c |
| F(2)         | 0x7ffefe4          | 2       | 0x0040016c |
| F(3)         | 0x7ffefec          | 3       | 0x0040016c |
| F(4)         | 0x7ffeff4          | 4       | 0x004000b4 |

## Execução e verificação

Ambiente: MARS 4.5, pseudoinstruções e macros habilitadas, delayed branching desabilitado e configuração de memória padrão. O código não foi organizado para executar com delay slots habilitados. A entrada prevista é um inteiro, conforme a especificação do trabalho.

As execuções foram conferidas com os valores da tabela do enunciado. Os valores esperados foram transcritos da especificação.

| Entradas, em ordem | Resultado obtido e esperado                                                   |
|--------------------|-------------------------------------------------------------------------------|
| 0, 1, 4, 19, -1    | Tabelas corretas para cada valor não negativo; encerramento em -1.            |
| -5                 | Cabeçalho e pedido de entrada, sem tabela; encerramento imediato.             |
| 4, 0, 4, -2        | Tabelas corretas, inclusive ao repetir 4 e voltar a zero; encerramento em -2. |

No MARS, \$sp terminou em 0x7ffeffc nos três cenários, igual ao valor inicial.

Para n = 19, os resultados conferidos são:

```
n      0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
F(n)   1 1 2 2 3 3 4 5 5 6 6 7 8 8 9 9 10 11 11 12
M(n)   0 0 1 2 2 3 4 4 5 6 6 7 7 8 9 9 10 11 11 12
```

A captura execucao-registradores.png mostra uma execução adicional na interface gráfica com 0, 4, 19 e -1. Após o encerramento, \$s0 = 0xffffffff (-1), \$s1 = 0x00000014 (20), \$v0 = 10 e \$sp = 0x7ffeffc. O contador 20 resulta da última tabela, que incluiu o índice 19.

A implementação recalcula subproblemas e não usa memoização. Valores maiores podem exigir muito mais tempo; a validação de resultados cobre n de 0 a 19.

## Material de entrega

O PDF reúne esta explicação, o algoritmo, o Assembly completo comentado e as sete capturas do MARS. As quatro vistas `codigo-01.png` a `codigo-04.png` cobrem o segmento de texto inteiro com sobreposição, de `0x00400000` a `0x004001c8` (115 instruções montadas). As demais capturas mostram a execução e os registradores finais, a pilha em Female e a pilha em Male.

O arquivo compactado contém somente o PDF e `hofstadter.asm`, como solicitado no enunciado. As imagens usadas nas capturas ficam incorporadas no PDF.

## Conferência dos requisitos

| Requisito                        | Evidência entregue                                                                           |
|----------------------------------|----------------------------------------------------------------------------------------------|
| Funções principal, Female e Male | <code>main</code> , <code>female</code> e <code>male</code> em <code>hofstadter.asm</code> . |
| Pelo menos três macros           | <code>print_string</code> , <code>print_int</code> e <code>read_int</code> .                 |
| Recursividade mútua              | Chamadas próprias e cruzadas em Female e Male.                                               |
| Preservação na pilha             | <code>n</code> e <code>\$ra</code> salvos e recuperados em quadros de 8 bytes.               |
| Cabeçalho inicial                | Título, data e três desenvolvedores na saída.                                                |
| Leitura e laço até negativo      | Leitura por <code>syscall 5</code> , teste de sinal e retorno à leitura.                     |
| Tabela de 0 até n                | Três linhas horizontais, verificadas até <code>n = 19</code> .                               |
| Código comentado                 | Assembly completo reproduzido neste documento.                                               |
| Algoritmo de alto nível          | Português estruturado incluído no PDF.                                                       |
| Área de código montada           | Quatro capturas que cobrem todas as instruções.                                              |
| Registradores finais e execução  | Captura da sessão com 0, 4, 19 e -1.                                                         |
| Pilha durante Female e Male      | Duas capturas com quadros ativos e explicação dos endereços.                                 |
| Documentação e Assembly separado | PDF e arquivo <code>.asm</code> incluídos no pacote compactado.                              |

# Algoritmo em alto nível

O programa recebe inteiros e apresenta as sequências de 0 até n, inclusive. Um valor negativo encerra o laço. A implementação em C está em `trabalho1.c`.

```
função Female(n)
    se n = 0
        retornar 1
    retornar n - Male(Female(n - 1))
fim função

função Male(n)
    se n = 0
        retornar 0
    retornar n - Female(Male(n - 1))
fim função

função main()
    mostrar "Sequencias Female e Male de Hofstadter - 28/09/2026"
    mostrar "Desenvolvedores: Juarez Fernando Goncalves dos Santos, Theo Carvalho Kirsch e Allan Rosa"

    repetir
        mostrar "Digite n para calcular F(n) e M(n) ou numero negativo para abortar a execucao: "
        ler inteiro n
        se n < 0
            encerrar

        mostrar "n", sem mudar de linha
        para i de 0 até n, inclusive
            mostrar i, separado por tabulação
        mudar de linha

        mostrar "F(n)", sem mudar de linha
        para i de 0 até n, inclusive
            mostrar Female(i), separado por tabulação
        mudar de linha

        mostrar "M(n)", sem mudar de linha
        para i de 0 até n, inclusive
            mostrar Male(i), separado por tabulação
        mudar de linha
    fim repetir
fim função
```

As chamadas são mutuamente recursivas. Female calcula Female(n - 1) e passa o resultado para Male; Male calcula Male(n - 1) e passa o resultado para Female. O parâmetro original n precisa continuar disponível para a subtração após essas chamadas.

# Implementação Assembly MIPS

Conteúdo integral de hofstadter.asm. Os números à esquerda indicam as linhas do arquivo.

```
1 # Trabalho 01 - Sequencias Female e Male de Hofstadter
2 # Disciplina: Organizacao e Arquitetura de Processadores
3 # Desenvolvedores: Juarez Fernando Goncalves dos Santos, Theo Carvalho Kirsch e Allan Rosa
4 #
5 # Implementacao equivalente ao algoritmo em C usando recursividade,
6 # macros e pilha para preservar o endereco de retorno e o parametro n.
7
8 .data
9 titulo: .asciiz "Sequencias Female e Male de Hofstadter - 28/09/2026\n"
10 autores: .asciiz "Desenvolvedores: Juarez Fernando Goncalves dos Santos, Theo Carvalho Kirsch e Allan Rosa\n"
11 prompt: .asciiz "\nDigite n para calcular F(n) e M(n) ou numero negativo para abortar a execucao: "
12 linha_n: .asciiz "n\t"
13 linha_f: .asciiz "F(n)\t"
14 linha_m: .asciiz "M(n)\t"
15 tab: .asciiz "\t"
16 nova_linha: .asciiz "\n"
17
18 # Macro para imprimir uma string
19 .macro print_string(%rotulo)
20     li $v0, 4          # codigo do syscall para imprimir string
21     la $a0, %rotulo    # endereco da string
22     syscall
23 .end_macro
24
25 # Macro para imprimir um inteiro
26 .macro print_int(%reg)
27     li $v0, 1          # codigo do syscall para imprimir inteiro
28     move $a0, %reg      # valor que sera impresso
29     syscall
30 .end_macro
31
32 # Macro para ler um inteiro
33 .macro read_int(%reg)
34     li $v0, 5          # codigo do syscall para ler inteiro
35     syscall
36     move %reg, $v0      # guarda o valor lido
37 .end_macro
38
39 .text
40 .globl main
41
42 main:
43     print_string(titulo) # exibe o titulo
44     print_string(autores) # exibe os desenvolvedores
45
46 leitura:
47     print_string(prompt) # solicita o valor de n
48     read_int($s0)        # $s0 guarda n
49
50     bltz $s0, fim         # numero negativo encerra o programa
51
52     # Mostra a primeira linha: n 0 1 2 ... n
53     print_string(linha_n)
54     li $s1, 0            # contador inicia em zero
55
56 imprime_n:
57     bgt $s1, $s0, fim_n  # termina quando contador > n
58     print_int($s1)       # imprime o contador
59     print_string(tab)
60     addi $s1, $s1, 1     # contador++
61     j imprime_n
62
63 fim_n:
```

```

64     print_string(nova_linha)
65
66     # Mostra a segunda linha: F(n)
67     print_string(linha_f)
68     li $s1, 0          # volta o contador para zero
69
70 imprime_f:
71     bgt $s1, $s0, fim_f # termina quando contador > n
72     move $a0, $s1       # passa o contador como parametro
73     jal female          # calcula F(contador)
74     move $t0, $v0       # guarda o resultado retornado
75     print_int($t0)      # imprime F(contador)
76     print_string(tab)
77     addi $s1, $s1, 1    # contador++
78     j imprime_f
79
80 fim_f:
81     print_string(nova_linha)
82
83     # Mostra a terceira linha: M(n)
84     print_string(linha_m)
85     li $s1, 0          # volta o contador para zero
86
87 imprime_m:
88     bgt $s1, $s0, fim_m # termina quando contador > n
89     move $a0, $s1       # passa o contador como parametro
90     jal male            # calcula M(contador)
91     move $t0, $v0       # guarda o resultado retornado
92     print_int($t0)      # imprime M(contador)
93     print_string(tab)
94     addi $s1, $s1, 1    # contador++
95     j imprime_m
96
97 fim_m:
98     print_string(nova_linha)
99     j leitura          # volta para uma nova entrada
100
101 fim:
102     li $v0, 10          # codigo do syscall para encerrar
103     syscall
104
105 # Entrada: $a0 = n >= 0; saida: $v0 = resultado.
106 # $a0 e $t0 sao temporarios; $s0 e $s1 permanecem intactos.
107 # F(n) = 1, se n = 0
108 # F(n) = n - M(F(n - 1)), se n > 0
109 female:
110     beq $a0, $zero, female_base # caso base F(0)
111
112     addi $sp, $sp, -8      # reserva duas palavras na pilha
113     sw $a0, 0($sp)        # salva o n original
114     sw $ra, 4($sp)        # salva o endereco de retorno
115
116     addi $a0, $a0, -1      # prepara n - 1
117     jal female            # calcula F(n - 1)
118
119     move $a0, $v0          # passa F(n - 1) para Male
120     jal male              # calcula M(F(n - 1))
121
122     lw $t0, 0($sp)        # recupera o n original
123     sub $v0, $t0, $v0     # retorna n - M(F(n - 1))
124
125     lw $ra, 4($sp)        # recupera o endereco de retorno
126     addi $sp, $sp, 8      # libera o espaco usado na pilha
127     jr $ra                # retorna para quem chamou
128
129 female_base:
130     li $v0, 1             # F(0) = 1
131     jr $ra
132
133 # M(n) = 0, se n = 0
134 # M(n) = n - F(M(n - 1)), se n > 0

```

```

135 male:
136     beq $a0, $zero, male_base # caso base M(0)
137
138     addi $sp, $sp, -8          # reserva duas palavras na pilha
139     sw $a0, 0($sp)             # salva o n original
140     sw $ra, 4($sp)             # salva o endereço de retorno
141
142     addi $a0, $a0, -1          # prepara n - 1
143     jal male                    # calcula M(n - 1)
144
145     move $a0, $v0              # passa M(n - 1) para Female
146     jal female                 # calcula F(M(n - 1))
147
148     lw $t0, 0($sp)             # recupera o n original
149     sub $v0, $t0, $v0          # retorna n - F(M(n - 1))
150
151     lw $ra, 4($sp)             # recupera o endereço de retorno
152     addi $sp, $sp, 8           # libera o espaço usado na pilha
153     jr $ra                     # retorna para quem chamou
154
155 male_base:
156     li $v0, 0                  # M(0) = 0
157     jr $ra

```

## Código montado - vista 1 de 4

Início do programa, cabeçalho, leitura e impressão dos índices. Montagem concluída sem erros.

/home/santojuca/repos/mips-hofstadter-trabalho01/hofstadter.asm - MARS 4.5 - Hofstadter

Run speed at max (no interaction)

Edit
Execute

Text Segment

| Bkpt                     | Address    | Code       | Basic                     | Source                                                        |
|--------------------------|------------|------------|---------------------------|---------------------------------------------------------------|
| <input type="checkbox"/> | 0x00400000 | 0x24020004 | addiu \$2,\$0,0x00000004  | 43: <20> li \$v0, 4 # codigo do syscall para imprimir string  |
| <input type="checkbox"/> | 0x00400004 | 0x3c011001 | lui \$1,0x00001001        | <21> la \$a0, titulo # endereco da string                     |
| <input type="checkbox"/> | 0x00400008 | 0x34240000 | ori \$4,\$1,0x00000000    |                                                               |
| <input type="checkbox"/> | 0x0040000c | 0x0000000c | syscall                   | <22> syscall                                                  |
| <input type="checkbox"/> | 0x00400010 | 0x24020004 | addiu \$2,\$0,0x00000004  | 44: <20> li \$v0, 4 # codigo do syscall para imprimir string  |
| <input type="checkbox"/> | 0x00400014 | 0x3c011001 | lui \$1,0x00001001        | <21> la \$a0, autores # endereco da string                    |
| <input type="checkbox"/> | 0x00400018 | 0x34240035 | ori \$4,\$1,0x00000035    |                                                               |
| <input type="checkbox"/> | 0x0040001c | 0x0000000c | syscall                   | <22> syscall                                                  |
| <input type="checkbox"/> | 0x00400020 | 0x24020004 | addiu \$2,\$0,0x00000004  | 47: <20> li \$v0, 4 # codigo do syscall para imprimir string  |
| <input type="checkbox"/> | 0x00400024 | 0x3c011001 | lui \$1,0x00001001        | <21> la \$a0, prompt # endereco da string                     |
| <input type="checkbox"/> | 0x00400028 | 0x3424008f | ori \$4,\$1,0x0000008f    |                                                               |
| <input type="checkbox"/> | 0x0040002c | 0x0000000c | syscall                   | <22> syscall                                                  |
| <input type="checkbox"/> | 0x00400030 | 0x24020005 | addiu \$2,\$0,0x00000005  | 48: <34> li \$v0, 5 # codigo do syscall para ler inteiro      |
| <input type="checkbox"/> | 0x00400034 | 0x0000000c | syscall                   | <35> syscall                                                  |
| <input type="checkbox"/> | 0x00400038 | 0x00028021 | addu \$16,\$0,\$2         | <36> move \$s0, \$v0 # guarda o valor lido                    |
| <input type="checkbox"/> | 0x0040003c | 0x06000043 | bltz \$16,0x00000043      | 50: bltz \$s0, fim # numero negativo encerra o programa       |
| <input type="checkbox"/> | 0x00400040 | 0x24020004 | addiu \$2,\$0,0x00000004  | 53: <20> li \$v0, 4 # codigo do syscall para imprimir string  |
| <input type="checkbox"/> | 0x00400044 | 0x3c011001 | lui \$1,0x00001001        | <21> la \$a0, linha_n # endereco da string                    |
| <input type="checkbox"/> | 0x00400048 | 0x342400e0 | ori \$4,\$1,0x000000e0    |                                                               |
| <input type="checkbox"/> | 0x0040004c | 0x0000000c | syscall                   | <22> syscall                                                  |
| <input type="checkbox"/> | 0x00400050 | 0x24110000 | addiu \$17,\$0,0x00000000 | 54: li \$s1, 0 # contador inicia em zero                      |
| <input type="checkbox"/> | 0x00400054 | 0x0211082a | slt \$1,\$16,\$17         | 57: bgt \$s1, \$s0, fim_n # termina quando contador > n       |
| <input type="checkbox"/> | 0x00400058 | 0x14200009 | bne \$1,\$0,0x00000009    |                                                               |
| <input type="checkbox"/> | 0x0040005c | 0x24020001 | addiu \$2,\$0,0x00000001  | 58: <27> li \$v0, 1 # codigo do syscall para imprimir inteiro |
| <input type="checkbox"/> | 0x00400060 | 0x00112021 | addu \$4,\$0,\$17         | <28> move \$a0, \$s1 # valor que sera impresso                |
| <input type="checkbox"/> | 0x00400064 | 0x0000000c | syscall                   | <29> syscall                                                  |
| <input type="checkbox"/> | 0x00400068 | 0x24020004 | addiu \$2,\$0,0x00000004  | 59: <20> li \$v0, 4 # codigo do syscall para imprimir string  |
| <input type="checkbox"/> | 0x0040006c | 0x3c011001 | lui \$1,0x00001001        | <21> la \$a0, tab # endereco da string                        |
| <input type="checkbox"/> | 0x00400070 | 0x342400ef | ori \$4,\$1,0x000000ef    |                                                               |
| <input type="checkbox"/> | 0x00400074 | 0x0000000c | syscall                   | <22> syscall                                                  |
| <input type="checkbox"/> | 0x00400078 | 0x22310001 | addi \$17,\$17,0x00000001 | 60: addi \$s1, \$s1, 1 # contador++                           |
| <input type="checkbox"/> | 0x0040007c | 0x08100015 | j 0x00400054              | 61: j imprime_n                                               |
| <input type="checkbox"/> | 0x00400080 | 0x24020004 | addiu \$2,\$0,0x00000004  | 64: <20> li \$v0, 4 # codigo do syscall para imprimir string  |
| <input type="checkbox"/> | 0x00400084 | 0x3c011001 | lui \$1,0x00001001        | <21> la \$a0, nova_linha # endereco da string                 |
| <input type="checkbox"/> | 0x00400088 | 0x342400f1 | ori \$4,\$1,0x000000f1    |                                                               |
| <input type="checkbox"/> | 0x0040008c | 0x0000000c | syscall                   | <22> syscall                                                  |

Data Segment

| Address    | Value (+0) | Value (+4) | Value (+8) | Value (+c) | Value (+10) | Value (+14) | Value (+18) | Value (+1c) |
|------------|------------|------------|------------|------------|-------------|-------------|-------------|-------------|
| 0x10010000 | 0x75716553 | 0x69636e65 | 0x46207361 | 0x6c616d65 | 0x20652065  | 0x656c616d  | 0x20656420  | 0x73666f48  |

0x10010000 (.data)

☒ Hexadecimal Addresses
 ☒ Hexadecimal Values
 ☐ ASCII



## Código montado - vista 2 de 4

Continuação do segmento de texto. As vistas se sobrepõem para manter a continuidade.

/home/santojuca/repos/mips-hofstadter-trabalho01/hofstadter.asm - MARS 4.5 - Hofstadter
Run speed at max (no interaction)

Edit
Execute

Text Segment
↗

| Bkpt                     | Address    | Code       | Basic                     | Source                                                        |
|--------------------------|------------|------------|---------------------------|---------------------------------------------------------------|
| <input type="checkbox"/> | 0x0040008c | 0x0000000c | syscall                   | <22> syscall                                                  |
| <input type="checkbox"/> | 0x00400090 | 0x24020004 | addiu \$2,\$0,0x00000004  | 67: <20> li \$v0, 4 # codigo do syscall para imprimir string  |
| <input type="checkbox"/> | 0x00400094 | 0x3c011001 | lui \$1,0x00001001        | <21> la \$a0, linha_f # endereco da string                    |
| <input type="checkbox"/> | 0x00400098 | 0x342400e3 | ori \$4,\$1,0x000000e3    |                                                               |
| <input type="checkbox"/> | 0x0040009c | 0x0000000c | syscall                   | <22> syscall                                                  |
| <input type="checkbox"/> | 0x004000a0 | 0x24110000 | addiu \$17,\$0,0x00000000 | 68: li \$s1, 0 # volta o contador para zero                   |
| <input type="checkbox"/> | 0x004000a4 | 0x0211082a | slt \$1,\$16,\$17         | 71: bgt \$s1, \$s0, fim_f # termina quando contador > n       |
| <input type="checkbox"/> | 0x004000a8 | 0x1420000c | bne \$1,\$0,0x0000000c    |                                                               |
| <input type="checkbox"/> | 0x004000ac | 0x00112021 | addu \$4,\$0,\$17         | 72: move \$a0, \$s1 # passa o contador como parametro         |
| <input type="checkbox"/> | 0x004000b0 | 0x0c100055 | jal 0x00400154            | 73: jal female # calcula F(contador)                          |
| <input type="checkbox"/> | 0x004000b4 | 0x00024021 | addu \$8,\$0,\$2          | 74: move \$t0, \$v0 # guarda o resultado retornado            |
| <input type="checkbox"/> | 0x004000b8 | 0x24020001 | addiu \$2,\$0,0x00000001  | 75: <27> li \$v0, 1 # codigo do syscall para imprimir inteiro |
| <input type="checkbox"/> | 0x004000bc | 0x00082021 | addu \$4,\$0,\$8          | <28> move \$a0, \$t0 # valor que sera impresso                |
| <input type="checkbox"/> | 0x004000c0 | 0x0000000c | syscall                   | <29> syscall                                                  |
| <input type="checkbox"/> | 0x004000c4 | 0x24020004 | addiu \$2,\$0,0x00000004  | 76: <20> li \$v0, 4 # codigo do syscall para imprimir string  |
| <input type="checkbox"/> | 0x004000c8 | 0x3c011001 | lui \$1,0x00001001        | <21> la \$a0, tab # endereco da string                        |
| <input type="checkbox"/> | 0x004000cc | 0x342400ef | ori \$4,\$1,0x000000ef    |                                                               |
| <input type="checkbox"/> | 0x004000d0 | 0x0000000c | syscall                   | <22> syscall                                                  |
| <input type="checkbox"/> | 0x004000d4 | 0x22310001 | addi \$17,\$17,0x00000001 | 77: addi \$s1, \$s1, 1 # contador++                           |
| <input type="checkbox"/> | 0x004000d8 | 0x08100029 | j 0x004000a4              | 78: j imprime_f                                               |
| <input type="checkbox"/> | 0x004000dc | 0x24020004 | addiu \$2,\$0,0x00000004  | 81: <20> li \$v0, 4 # codigo do syscall para imprimir string  |
| <input type="checkbox"/> | 0x004000e0 | 0x3c011001 | lui \$1,0x00001001        | <21> la \$a0, nova_linha # endereco da string                 |
| <input type="checkbox"/> | 0x004000e4 | 0x342400f1 | ori \$4,\$1,0x000000f1    |                                                               |
| <input type="checkbox"/> | 0x004000e8 | 0x0000000c | syscall                   | <22> syscall                                                  |
| <input type="checkbox"/> | 0x004000ec | 0x24020004 | addiu \$2,\$0,0x00000004  | 84: <20> li \$v0, 4 # codigo do syscall para imprimir string  |
| <input type="checkbox"/> | 0x004000f0 | 0x3c011001 | lui \$1,0x00001001        | <21> la \$a0, linha_m # endereco da string                    |
| <input type="checkbox"/> | 0x004000f4 | 0x342400e9 | ori \$4,\$1,0x000000e9    |                                                               |
| <input type="checkbox"/> | 0x004000f8 | 0x0000000c | syscall                   | <22> syscall                                                  |
| <input type="checkbox"/> | 0x004000fc | 0x24110000 | addiu \$17,\$0,0x00000000 | 85: li \$s1, 0 # volta o contador para zero                   |
| <input type="checkbox"/> | 0x00400100 | 0x0211082a | slt \$1,\$16,\$17         | 88: bgt \$s1, \$s0, fim_m # termina quando contador > n       |
| <input type="checkbox"/> | 0x00400104 | 0x1420000c | bne \$1,\$0,0x0000000c    |                                                               |
| <input type="checkbox"/> | 0x00400108 | 0x00112021 | addu \$4,\$0,\$17         | 89: move \$a0, \$s1 # passa o contador como parametro         |
| <input type="checkbox"/> | 0x0040010c | 0x0c100064 | jal 0x00400190            | 90: jal male # calcula M(contador)                            |
| <input type="checkbox"/> | 0x00400110 | 0x00024021 | addu \$8,\$0,\$2          | 91: move \$t0, \$v0 # guarda o resultado retornado            |
| <input type="checkbox"/> | 0x00400114 | 0x24020001 | addiu \$2,\$0,0x00000001  | 92: <27> li \$v0, 1 # codigo do syscall para imprimir inteiro |
| <input type="checkbox"/> | 0x00400118 | 0x00082021 | addu \$4,\$0,\$8          | <28> move \$a0, \$t0 # valor que sera impresso                |

Data Segment
↗

| Address    | Value (+0) | Value (+4) | Value (+8) | Value (+c) | Value (+10) | Value (+14) | Value (+18) | Value (+1c) |
|------------|------------|------------|------------|------------|-------------|-------------|-------------|-------------|
| 0x10010000 | 0x75716553 | 0x69636e65 | 0x46207361 | 0x6c616d65 | 0x20652065  | 0x656c614d  | 0x20656420  | 0x73666f48  |

←
→

0x10010000 (.data)
▼

☒ Hexadecimal Addresses
 ☒ Hexadecimal Values
 ☐ ASCII

Mars Messages
Run I/O

Assemble: assembling /home/santojuca/repos/mips-hofstadter-trabalho01/hofstadter.asm  
  
 Assemble: operation completed successfully.

Clear

Registers
Coproc 1
Coproc 0

## Código montado - vista 3 de 4

Laços de impressão, encerramento e início das funções recursivas.

/home/santojuca/repos/mips-hofstadter-trabalho01/hofstadter.asm - MARS 4.5 - Hofstadter

EditExecute

Text Segment

| Bkpt                     | Address    | Code       | Basic                       | Source                                                       |
|--------------------------|------------|------------|-----------------------------|--------------------------------------------------------------|
| <input type="checkbox"/> | 0x00400118 | 0x00082021 | addu \$4,\$0,\$8            | <28> move \$a0, \$t0 # valor que sera impresso               |
| <input type="checkbox"/> | 0x0040011c | 0x0000000c | syscall                     | <29> syscall                                                 |
| <input type="checkbox"/> | 0x00400120 | 0x24020004 | addiu \$2,\$0,0x00000004    | 93: <20> li \$v0, 4 # codigo do syscall para imprimir string |
| <input type="checkbox"/> | 0x00400124 | 0x3c011001 | lui \$1,0x00001001          | <21> la \$a0, tab # endereco da string                       |
| <input type="checkbox"/> | 0x00400128 | 0x342400ef | ori \$4,\$1,0x000000ef      |                                                              |
| <input type="checkbox"/> | 0x0040012c | 0x0000000c | syscall                     | <22> syscall                                                 |
| <input type="checkbox"/> | 0x00400130 | 0x22310001 | addi \$l7,\$l7,0x00000001   | 94: addi \$s1, \$s1, 1 # contador++                          |
| <input type="checkbox"/> | 0x00400134 | 0x08100040 | j 0x00400100                | 95: j imprime_m                                              |
| <input type="checkbox"/> | 0x00400138 | 0x24020004 | addiu \$2,\$0,0x00000004    | 98: <20> li \$v0, 4 # codigo do syscall para imprimir string |
| <input type="checkbox"/> | 0x0040013c | 0x3c011001 | lui \$1,0x00001001          | <21> la \$a0, nova_linha # endereco da string                |
| <input type="checkbox"/> | 0x00400140 | 0x342400f1 | ori \$4,\$1,0x000000f1      |                                                              |
| <input type="checkbox"/> | 0x00400144 | 0x0000000c | syscall                     | <22> syscall                                                 |
| <input type="checkbox"/> | 0x00400148 | 0x08100008 | j 0x00400020                | 99: j leitura # volta para uma nova entrada                  |
| <input type="checkbox"/> | 0x0040014c | 0x2402000a | addiu \$2,\$0,0x0000000a    | 102: li \$v0, 10 # codigo do syscall para encerrar           |
| <input type="checkbox"/> | 0x00400150 | 0x0000000c | syscall                     | 103: syscall                                                 |
| <input type="checkbox"/> | 0x00400154 | 0x1080000c | beq \$4,\$0,0x0000000c      | 110: beq \$a0, \$zero, female_base # caso base F(0)          |
| <input type="checkbox"/> | 0x00400158 | 0x23bdfff8 | addi \$29,\$29,0xfffffffff8 | 112: addi \$sp, \$sp, -8 # reserva duas palavras na pilha    |
| <input type="checkbox"/> | 0x0040015c | 0xafaf0000 | sw \$4,0x00000000(\$29)     | 113: sw \$a0, 0(\$sp) # salva o n original                   |
| <input type="checkbox"/> | 0x00400160 | 0xafbf0004 | sw \$31,0x00000004(\$29)    | 114: sw \$ra, 4(\$sp) # salva o endereco de retorno          |
| <input type="checkbox"/> | 0x00400164 | 0x2084ffff | addi \$4,\$4,0xfffffffff    | 116: addi \$a0, \$a0, -1 # prepara n - 1                     |
| <input type="checkbox"/> | 0x00400168 | 0x0c100055 | jal 0x00400154              | 117: jal female # calcula F(n - 1)                           |
| <input type="checkbox"/> | 0x0040016c | 0x00022021 | addu \$4,\$0,\$2            | 119: move \$a0, \$v0 # passa F(n - 1) para Male              |
| <input type="checkbox"/> | 0x00400170 | 0x0c100064 | jal 0x00400190              | 120: jal male # calcula M(F(n - 1))                          |
| <input type="checkbox"/> | 0x00400174 | 0x8fa80000 | lw \$8,0x00000000(\$29)     | 122: lw \$t0, 0(\$sp) # recupera o n original                |
| <input type="checkbox"/> | 0x00400178 | 0x01021022 | sub \$2,\$8,\$2             | 123: sub \$v0, \$t0, \$v0 # retorna n - M(F(n - 1))          |
| <input type="checkbox"/> | 0x0040017c | 0x8fbf0004 | lw \$31,0x00000004(\$29)    | 125: lw \$ra, 4(\$sp) # recupera o endereco de retorno       |
| <input type="checkbox"/> | 0x00400180 | 0x23bd0008 | addi \$29,\$29,0x00000008   | 126: addi \$sp, \$sp, 8 # libera o espaco usado na pilha     |
| <input type="checkbox"/> | 0x00400184 | 0x03e00008 | jr \$31                     | 127: jr \$ra # retorna para quem chamou                      |
| <input type="checkbox"/> | 0x00400188 | 0x24020001 | addiu \$2,\$0,0x00000001    | 130: li \$v0, 1 # F(0) = 1                                   |
| <input type="checkbox"/> | 0x0040018c | 0x03e00008 | jr \$31                     | 131: jr \$ra                                                 |
| <input type="checkbox"/> | 0x00400190 | 0x1080000c | beq \$4,\$0,0x0000000c      | 136: beq \$a0, \$zero, male_base # caso base M(0)            |
| <input type="checkbox"/> | 0x00400194 | 0x23bdfff8 | addi \$29,\$29,0xfffffffff8 | 138: addi \$sp, \$sp, -8 # reserva duas palavras na pilha    |
| <input type="checkbox"/> | 0x00400198 | 0xafaf0000 | sw \$4,0x00000000(\$29)     | 139: sw \$a0, 0(\$sp) # salva o n original                   |
| <input type="checkbox"/> | 0x0040019c | 0xafbf0004 | sw \$31,0x00000004(\$29)    | 140: sw \$ra, 4(\$sp) # salva o endereco de retorno          |
| <input type="checkbox"/> | 0x004001a0 | 0x2084ffff | addi \$4,\$4,0xfffffffff    | 142: addi \$a0, \$a0, -1 # prepara n - 1                     |
| <input type="checkbox"/> | 0x004001a4 | 0x0c100064 | jal 0x00400190              | 143: jal male # calcula M(n - 1)                             |

Data Segment

| Address    | Value (+0) | Value (+4) | Value (+8) | Value (+c) | Value (+10) | Value (+14) | Value (+18) | Value (+1c) |
|------------|------------|------------|------------|------------|-------------|-------------|-------------|-------------|
| 0x10010000 | 0x75716553 | 0x69636e65 | 0x46207361 | 0x6c616d65 | 0x20652065  | 0x656c614d  | 0x20656420  | 0x73666f48  |

←→

0x10010000 (.data)

☒ Hexadecimal Addresses☒ Hexadecimal Values☐ ASCII

Mars MessagesRun I/O

Assemble: assembling /home/santojuca/repos/mips-hofstadter-trabalho01/hofstadter.asm  
Assemble: operation completed successfully.  

Clear



## Código montado - vista 4 de 4

Final das funções Female e Male. Última instrução no endereço 0x004001c8.

/home/santojuca/repos/mips-hofstadter-trabalho01/hofstadter.asm - MARS 4.5 - Hofstadter

EditExecute

Run speed at max (no interaction)

Text Segment

| Bkpt | Address    | Code       | Basic                      | Source                                                    |
|------|------------|------------|----------------------------|-----------------------------------------------------------|
|      | 0x0040013c | 0x3c011001 | lui \$1,0x00001001         | <21> la \$a0, nova_linha # endereco da string             |
|      | 0x00400140 | 0x342400f1 | ori \$4,\$1,0x000000f1     |                                                           |
|      | 0x00400144 | 0x0000000c | syscall                    | <22> syscall                                              |
|      | 0x00400148 | 0x08100008 | j 0x00400020               | 99: j leitura # volta para uma nova entrada               |
|      | 0x0040014c | 0x2402000a | addiu \$2,\$0,0x0000000a   | 102: li \$v0, 10 # codigo do syscall para encerrar        |
|      | 0x00400150 | 0x0000000c | syscall                    | 103: syscall                                              |
|      | 0x00400154 | 0x1080000c | beq \$4,\$0,0x0000000c     | 110: beq \$a0, \$zero, female_base # caso base F(0)       |
|      | 0x00400158 | 0x23bdfff8 | addi \$29,\$29,0xffffffff8 | 112: addi \$sp, \$sp, -8 # reserva duas palavras na pilha |
|      | 0x0040015c | 0xa0400000 | sw \$4,0x00000000(\$29)    | 113: sw \$a0, 0(\$sp) # salva o n original                |
|      | 0x00400160 | 0xafbf0004 | sw \$31,0x00000004(\$29)   | 114: sw \$ra, 4(\$sp) # salva o endereco de retorno       |
|      | 0x00400164 | 0x2084ffff | addi \$4,\$4,0xfffffffff   | 116: addi \$a0, \$a0, -1 # prepara n - 1                  |
|      | 0x00400168 | 0x0c100055 | jal 0x00400154             | 117: jal female # calcula F(n - 1)                        |
|      | 0x0040016c | 0x00022021 | addu \$4,\$0,\$2           | 119: move \$a0, \$v0 # passa F(n - 1) para Male           |
|      | 0x00400170 | 0x0c100064 | jal 0x00400190             | 120: jal male # calcula M(F(n - 1))                       |
|      | 0x00400174 | 0x8fa80000 | lw \$8,0x00000000(\$29)    | 122: lw \$t0, 0(\$sp) # recupera o n original             |
|      | 0x00400178 | 0x01021022 | sub \$2,\$8,\$2            | 123: sub \$v0, \$t0, \$v0 # retorna n - M(F(n - 1))       |
|      | 0x0040017c | 0x8fbf0004 | lw \$31,0x00000004(\$29)   | 125: lw \$ra, 4(\$sp) # recupera o endereco de retorno    |
|      | 0x00400180 | 0x23bd0008 | addi \$29,\$29,0x00000008  | 126: addi \$sp, \$sp, 8 # libera o espaco usado na pilha  |
|      | 0x00400184 | 0x03e00008 | jr \$31                    | 127: jr \$ra # retorna para quem chamou                   |
|      | 0x00400188 | 0x24020001 | addiu \$2,\$0,0x00000001   | 130: li \$v0, 1 # F(0) = 1                                |
|      | 0x0040018c | 0x03e00008 | jr \$31                    | 131: jr \$ra                                              |
|      | 0x00400190 | 0x1080000c | beq \$4,\$0,0x0000000c     | 136: beq \$a0, \$zero, male_base # caso base M(0)         |
|      | 0x00400194 | 0x23bdfff8 | addi \$29,\$29,0xffffffff8 | 138: addi \$sp, \$sp, -8 # reserva duas palavras na pilha |
|      | 0x00400198 | 0xa0400000 | sw \$4,0x00000000(\$29)    | 139: sw \$a0, 0(\$sp) # salva o n original                |
|      | 0x0040019c | 0xafbf0004 | sw \$31,0x00000004(\$29)   | 140: sw \$ra, 4(\$sp) # salva o endereco de retorno       |
|      | 0x004001a0 | 0x2084ffff | addi \$4,\$4,0xfffffffff   | 142: addi \$a0, \$a0, -1 # prepara n - 1                  |
|      | 0x004001a4 | 0x0c100064 | jal 0x00400190             | 143: jal male # calcula M(n - 1)                          |
|      | 0x004001a8 | 0x00022021 | addu \$4,\$0,\$2           | 145: move \$a0, \$v0 # passa M(n - 1) para Female         |
|      | 0x004001ac | 0x0c100055 | jal 0x00400154             | 146: jal female # calcula F(M(n - 1))                     |
|      | 0x004001b0 | 0x8fa80000 | lw \$8,0x00000000(\$29)    | 148: lw \$t0, 0(\$sp) # recupera o n original             |
|      | 0x004001b4 | 0x01021022 | sub \$2,\$8,\$2            | 149: sub \$v0, \$t0, \$v0 # retorna n - F(M(n - 1))       |
|      | 0x004001b8 | 0x8fbf0004 | lw \$31,0x00000004(\$29)   | 151: lw \$ra, 4(\$sp) # recupera o endereco de retorno    |
|      | 0x004001bc | 0x23bd0008 | addi \$29,\$29,0x00000008  | 152: addi \$sp, \$sp, 8 # libera o espaco usado na pilha  |
|      | 0x004001c0 | 0x03e00008 | jr \$31                    | 153: jr \$ra # retorna para quem chamou                   |
|      | 0x004001c4 | 0x24020000 | addiu \$2,\$0,0x00000000   | 156: li \$v0, 0 # M(0) = 0                                |
|      | 0x004001c8 | 0x03e00008 | jr \$31                    | 157: jr \$ra                                              |

Data Segment

| Address    | Value (+0) | Value (+4) | Value (+8) | Value (+c) | Value (+10) | Value (+14) | Value (+18) | Value (+1c) |
|------------|------------|------------|------------|------------|-------------|-------------|-------------|-------------|
| 0x10010000 | 0x75716553 | 0x69636e65 | 0x46207361 | 0x6c616d65 | 0x20652065  | 0x656c614d  | 0x20656420  | 0x73666f48  |

Hexadecimal Addresses Hexadecimal Values ASCII

Mars MessagesRun I/O  
Assemble: assembling /home/santojuca/repos/mips-hofstadter-trabalho01/hofstadter.asm  
Assemble: operation completed successfully.  
Clear

## Execução completa e registradores finais

Entradas 0, 4, 19 e -1. Ao encerrar: \$s0 = -1, \$s1 = 20, \$v0 = 10 e \$sp = 0x7ffffeffc.

/home/santojuca/repos/mips-hofstadter-trabalho01/hofstadter.asm - MARS 4.5 - Hofstadter
Run speed at max (no interaction)

Edit

Execute

Text Segment

| Bkpt                     | Address    | Code       | Basic                      | Source                                                    |
|--------------------------|------------|------------|----------------------------|-----------------------------------------------------------|
| <input type="checkbox"/> | 0x0040013c | 0x3c011001 | lui \$1,0x00001001         | <21> la \$a0, nova_linha # endereco da string             |
| <input type="checkbox"/> | 0x00400140 | 0x342400f1 | ori \$4,\$1,0x000000f1     |                                                           |
| <input type="checkbox"/> | 0x00400144 | 0x0000000c | syscall                    | <22> syscall                                              |
| <input type="checkbox"/> | 0x00400148 | 0x08100008 | j 0x00400020               | 99: j leitura # volta para uma nova entrada               |
| <input type="checkbox"/> | 0x0040014c | 0x2402000a | addiu \$2,\$0,0x0000000a   | 102: li \$v0, 10 # codigo do syscall para encerrar        |
| <input type="checkbox"/> | 0x00400150 | 0x0000000c | syscall                    | 103: syscall                                              |
| <input type="checkbox"/> | 0x00400154 | 0x1080000c | beq \$4,\$0,0x0000000c     | 110: beq \$a0, \$zero, female_base # caso base F(0)       |
| <input type="checkbox"/> | 0x00400158 | 0x23bdfbf8 | addi \$29,\$29,0xffffffff8 | 112: addi \$sp, \$sp, -8 # reserva duas palavras na pilha |
| <input type="checkbox"/> | 0x0040015c | 0xafa40000 | sw \$4,0x00000000(\$29)    | 113: sw \$a0, 0(\$sp) # salva o n original                |
| <input type="checkbox"/> | 0x00400160 | 0xafbf0004 | sw \$31,0x00000004(\$29)   | 114: sw \$ra, 4(\$sp) # salva o endereco de retorno       |
| <input type="checkbox"/> | 0x00400164 | 0x2084ffff | addi \$4,\$4,0xffffffff    | 116: addi \$a0, \$a0, -1 # prepara n - 1                  |
| <input type="checkbox"/> | 0x00400168 | 0x0c100055 | jal 0x00400154             | 117: jal female # calcula F(n - 1)                        |
| <input type="checkbox"/> | 0x0040016c | 0x00022021 | addu \$4,\$0,\$2           | 119: move \$a0, \$v0 # passa F(n - 1) para Male           |
| <input type="checkbox"/> | 0x00400170 | 0x0c100064 | jal 0x00400190             | 120: jal male # calcula M(F(n - 1))                       |
| <input type="checkbox"/> | 0x00400174 | 0x8fa80000 | lw \$8,0x00000000(\$29)    | 122: lw \$t0, 0(\$sp) # recupera o n original             |
| <input type="checkbox"/> | 0x00400178 | 0x01021022 | sub \$2,\$8,\$2            | 123: sub \$v0, \$t0, \$v0 # retorna n - M(F(n - 1))       |
| <input type="checkbox"/> | 0x0040017c | 0x8fbf0004 | lw \$31,0x00000004(\$29)   | 125: lw \$ra, 4(\$sp) # recupera o endereco de retorno    |
| <input type="checkbox"/> | 0x00400180 | 0x23bd0008 | addi \$29,\$29,0x00000008  | 126: addi \$sp, \$sp, 8 # libera o espaco usado na pilha  |
| <input type="checkbox"/> | 0x00400184 | 0x03e00008 | jr \$31                    | 127: jr \$ra # retorna para quem chamou                   |
| <input type="checkbox"/> | 0x00400188 | 0x24020001 | addiu \$2,\$0,0x00000001   | 130: li \$v0, 1 # F(0) = 1                                |
| <input type="checkbox"/> | 0x0040018c | 0x03e00008 | jr \$31                    | 131: jr \$ra                                              |

Mars Messages

Run I/O

Sequencias Female e Male de Hofstadter - 28/09/2026

Desenvolvedores: Juarez Fernando Goncalves dos Santos, Theo Carvalho Kirsch e Allan Rosa

Digite n para calcular F(n) e M(n) ou numero negativo para abortar a execucao: 0  
 n 0  
 F(n) 1  
 M(n) 0

Digite n para calcular F(n) e M(n) ou numero negativo para abortar a execucao: 4  
 n 0 1 2 3 4  
 F(n) 1 1 2 2 3  
 M(n) 0 0 1 2 2

Digite n para calcular F(n) e M(n) ou numero negativo para abortar a execucao: 19  
 n 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19  
 F(n) 1 1 2 2 3 3 4 5 5 6 6 7 8 8 9 9 10 11 11 12  
 M(n) 0 0 1 2 2 3 4 4 5 6 6 7 7 8 9 9 10 11 11 12

Digite n para calcular F(n) e M(n) ou numero negativo para abortar a execucao: -1  
  
 -- program is finished running --

Registers

Coproc 1

Coproc 0

| Name   | Number | Value      |
|--------|--------|------------|
| \$zero | 0      | 0x00000000 |
| \$at   | 1      | 0x10010000 |
| \$v0   | 2      | 0x0000000a |
| \$v1   | 3      | 0x00000000 |
| \$a0   | 4      | 0x1001000f |
| \$a1   | 5      | 0x00000000 |
| \$a2   | 6      | 0x00000000 |
| \$a3   | 7      | 0x00000000 |
| \$t0   | 8      | 0x0000000c |
| \$t1   | 9      | 0x00000000 |
| \$t2   | 10     | 0x00000000 |
| \$t3   | 11     | 0x00000000 |
| \$t4   | 12     | 0x00000000 |
| \$t5   | 13     | 0x00000000 |
| \$t6   | 14     | 0x00000000 |
| \$t7   | 15     | 0x00000000 |
| \$s0   | 16     | 0xffffffff |
| \$s1   | 17     | 0x00000014 |
| \$s2   | 18     | 0x00000000 |
| \$s3   | 19     | 0x00000000 |
| \$s4   | 20     | 0x00000000 |
| \$s5   | 21     | 0x00000000 |
| \$s6   | 22     | 0x00000000 |
| \$s7   | 23     | 0x00000000 |
| \$t8   | 24     | 0x00000000 |
| \$t9   | 25     | 0x00000000 |
| \$k0   | 26     | 0x00000000 |
| \$k1   | 27     | 0x00000000 |
| \$gp   | 28     | 0x1000     |



Pilha durante Female

F(4) chamou F(3), que está prestes a chamar F(2). Dois quadros ativos: \$sp = 0x7ffefec.

Run speed at max (no interaction)

Edit

Execute

Text Segment

| Bkpt | Address    | Code       | Basic                      | Source                                                        |
|------|------------|------------|----------------------------|---------------------------------------------------------------|
|      | 0x00400114 | 0x24020001 | addiu \$2,\$0,0x00000001   | 92: <27> li \$v0, 1 # código do syscall para imprimir inteiro |
|      | 0x00400118 | 0x00082021 | addu \$4,\$0,\$8           | <28> move \$a0, \$t0 # valor que sera impresso                |
|      | 0x0040011c | 0x0000000c | syscall                    | <29> syscall                                                  |
|      | 0x00400120 | 0x24020004 | addiu \$2,\$0,0x00000004   | 93: <20> li \$v0, 4 # código do syscall para imprimir string  |
|      | 0x00400124 | 0x3c011001 | lui \$1,0x00001001         | <21> la \$a0, tab # endereço da string                        |
|      | 0x00400128 | 0x342400ef | ori \$4,\$1,0x000000ef     |                                                               |
|      | 0x0040012c | 0x0000000c | syscall                    | <22> syscall                                                  |
|      | 0x00400130 | 0x22310001 | addi \$17,\$17,0x00000001  | 94: addi \$s1, \$s1, 1 # contador++                           |
|      | 0x00400134 | 0x08100040 | j 0x00400100               | 95: j imprime_m                                               |
|      | 0x00400138 | 0x24020004 | addiu \$2,\$0,0x00000004   | 98: <20> li \$v0, 4 # código do syscall para imprimir string  |
|      | 0x0040013c | 0x3c011001 | lui \$1,0x00001001         | <21> la \$a0, nova_linha # endereço da string                 |
|      | 0x00400140 | 0x342400f1 | ori \$4,\$1,0x000000f1     |                                                               |
|      | 0x00400144 | 0x0000000c | syscall                    | <22> syscall                                                  |
|      | 0x00400148 | 0x08100008 | j 0x00400020               | 99: j leitura # volta para uma nova entrada                   |
|      | 0x0040014c | 0x2402000a | addiu \$2,\$0,0x0000000a   | 102: li \$v0, 10 # código do syscall para encerrar            |
|      | 0x00400150 | 0x0000000c | syscall                    | 103: syscall                                                  |
|      | 0x00400154 | 0x1080000c | beq \$4,\$0,0x0000000c     | 110: beq \$a0, \$zero, female_base # caso base F(0)           |
|      | 0x00400158 | 0x23bdfbf8 | addi \$29,\$29,0xffffffff8 | 112: addi \$sp, \$sp, -8 # reserva duas palavras na pilha     |
|      | 0x0040015c | 0xaf400000 | sw \$4,0x00000000(\$29)    | 113: sw \$a0, 0(\$sp) # salva o n original                    |
|      | 0x00400160 | 0xafbf0004 | sw \$31,0x00000004(\$29)   | 114: sw \$ra, 4(\$sp) # salva o endereço de retorno           |

Data Segment

| Address    | Value (+0) | Value (+4) | Value (+8) | Value (+c) | Value (+10) | Value (+14) | Value (+18) | Value (+1c) |
|------------|------------|------------|------------|------------|-------------|-------------|-------------|-------------|
| 0x7ffefe0  | 0x00400174 | 0x00000001 | 0x004001a8 | 0x00000003 | 0x0040016c  | 0x00000004  | 0x004000b4  | 0x00000000  |
| 0x7ffff00  | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff020 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff040 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff060 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff080 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff0a0 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff0c0 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff0e0 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff100 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff120 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff140 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff160 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff180 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff1a0 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff1c0 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |

Registers

Coproc 1

Coproc 0

| Name   | Number | Value      |
|--------|--------|------------|
| \$zero | 0      | 0x00000000 |
| \$at   | 1      | 0x00000000 |
| \$v0   | 2      | 0x00000004 |
| \$v1   | 3      | 0x00000000 |
| \$a0   | 4      | 0x00000002 |
| \$a1   | 5      | 0x00000000 |
| \$a2   | 6      | 0x00000000 |
| \$a3   | 7      | 0x00000000 |
| \$t0   | 8      | 0x00000002 |
| \$t1   | 9      | 0x00000000 |
| \$t2   | 10     | 0x00000000 |
| \$t3   | 11     | 0x00000000 |
| \$t4   | 12     | 0x00000000 |
| \$t5   | 13     | 0x00000000 |
| \$t6   | 14     | 0x00000000 |
| \$t7   | 15     | 0x00000000 |
| \$s0   | 16     | 0x00000004 |
| \$s1   | 17     | 0x00000004 |
| \$s2   | 18     | 0x00000000 |
| \$s3   | 19     | 0x00000000 |
| \$s4   | 20     | 0x00000000 |
| \$s5   | 21     | 0x00000000 |
| \$s6   | 22     | 0x00000000 |
| \$s7   | 23     | 0x00000000 |
| \$t8   | 24     | 0x00000000 |
| \$t9   | 25     | 0x00000000 |
| \$k0   | 26     | 0x00000000 |
| \$k1   | 27     | 0x00000000 |
| \$gp   | 28     | 0x10008000 |
| \$sp   | 29     | 0x7ffefec  |
| \$fp   | 30     | 0x00000000 |
| \$ra   | 31     | 0x0040016c |
| pc     |        | 0x00400168 |
| hi     |        | 0x00000000 |
| lo     |        | 0x00000000 |

Mars Messages

Run I/O

Go: execution completed successfully.

Go: running hofstadter.asm

Go: execution paused at breakpoint: hofstadter.asm

Go: running hofstadter.asm

Go: execution paused at breakpoint: hofstadter.asm

# Pilha durante Male

Cadeia ativa F(4), F(3), F(2), F(1), M(1). Cinco quadros (40 bytes): \$sp = 0x7ffefd4.

Run speed at max (no interaction)

Edit

Execute

Text Segment

| Bkpt                                | Address    | Code       | Basic                      | Source                                                    |
|-------------------------------------|------------|------------|----------------------------|-----------------------------------------------------------|
| <input type="checkbox"/>            | 0x00400158 | 0x230d1118 | addi \$29,\$29,0x11111118  | 112: addi \$sp, \$sp, -8 # reserva duas palavras na pilha |
| <input type="checkbox"/>            | 0x0040015c | 0xafa40000 | sw \$4,0x00000000(\$29)    | 113: sw \$a0, 0(\$sp) # salva o n original                |
| <input type="checkbox"/>            | 0x00400160 | 0xafbf0004 | sw \$31,0x00000004(\$29)   | 114: sw \$ra, 4(\$sp) # salva o endereco de retorno       |
| <input type="checkbox"/>            | 0x00400164 | 0x2084ffff | addi \$4,\$4,0xffffffff    | 116: addi \$a0, \$a0, -1 # prepara n - 1                  |
| <input type="checkbox"/>            | 0x00400168 | 0x0c100055 | jal 0x00400154             | 117: jal female # calcula F(n - 1)                        |
| <input type="checkbox"/>            | 0x0040016c | 0x00022021 | addu \$4,\$0,\$2           | 119: move \$a0, \$v0 # passa F(n - 1) para Male           |
| <input type="checkbox"/>            | 0x00400170 | 0x0c100064 | jal 0x00400190             | 120: jal male # calcula M(F(n - 1))                       |
| <input type="checkbox"/>            | 0x00400174 | 0x8fa80000 | lw \$8,0x00000000(\$29)    | 122: lw \$t0, 0(\$sp) # recupera o n original             |
| <input type="checkbox"/>            | 0x00400178 | 0x01021022 | sub \$2,\$8,\$2            | 123: sub \$v0, \$t0, \$v0 # retorna n - M(F(n - 1))       |
| <input type="checkbox"/>            | 0x0040017c | 0x8fbf0004 | lw \$31,0x00000004(\$29)   | 125: lw \$ra, 4(\$sp) # recupera o endereco de retorno    |
| <input type="checkbox"/>            | 0x00400180 | 0x23bd0008 | addi \$29,\$29,0x00000008  | 126: addi \$sp, \$sp, 8 # libera o espaco usado na pilha  |
| <input type="checkbox"/>            | 0x00400184 | 0x03e00008 | jr \$31                    | 127: jr \$ra # retorna para quem chamou                   |
| <input type="checkbox"/>            | 0x00400188 | 0x24020001 | addiu \$2,\$0,0x00000001   | 130: li \$v0, 1 # F(0) = 1                                |
| <input type="checkbox"/>            | 0x0040018c | 0x03e00008 | jr \$31                    | 131: jr \$ra                                              |
| <input type="checkbox"/>            | 0x00400190 | 0x1080000c | beq \$4,\$0,0x0000000c     | 136: beq \$a0, \$zero, male_base # caso base M(0)         |
| <input type="checkbox"/>            | 0x00400194 | 0x23bdfff8 | addi \$29,\$29,0xffffffff8 | 138: addi \$sp, \$sp, -8 # reserva duas palavras na pilha |
| <input type="checkbox"/>            | 0x00400198 | 0xafa40000 | sw \$4,0x00000000(\$29)    | 139: sw \$a0, 0(\$sp) # salva o n original                |
| <input type="checkbox"/>            | 0x0040019c | 0xafbf0004 | sw \$31,0x00000004(\$29)   | 140: sw \$ra, 4(\$sp) # salva o endereco de retorno       |
| <input type="checkbox"/>            | 0x004001a0 | 0x2084ffff | addi \$4,\$4,0xffffffff    | 142: addi \$a0, \$a0, -1 # prepara n - 1                  |
| <input checked="" type="checkbox"/> | 0x004001a4 | 0x0c100064 | jal 0x00400190             | 143: jal male # calcula M(n - 1)                          |

Data Segment

| Address    | Value (+0) | Value (+4) | Value (+8) | Value (+c) | Value (+10) | Value (+14) | Value (+18) | Value (+1c) |
|------------|------------|------------|------------|------------|-------------|-------------|-------------|-------------|
| 0x7ffefc0  | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000001  | 0x00400174  | 0x00000001  |
| 0x7ffefe0  | 0x0040016c | 0x00000002 | 0x0040016c | 0x00000003 | 0x0040016c  | 0x00000004  | 0x00400b4   | 0x00000000  |
| 0x7ffff00  | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff020 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff040 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff060 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff080 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff0a0 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff0c0 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff0e0 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff100 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff120 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff140 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff160 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff180 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x7ffff1a0 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |

Registers

Coproc 1

Coproc 0

| Name   | Number | Value      |
|--------|--------|------------|
| \$zero | 0      | 0x00000000 |
| \$at   | 1      | 0x00000000 |
| \$v0   | 2      | 0x00000001 |
| \$v1   | 3      | 0x00000000 |
| \$a0   | 4      | 0x00000000 |
| \$a1   | 5      | 0x00000000 |
| \$a2   | 6      | 0x00000000 |
| \$a3   | 7      | 0x00000000 |
| \$t0   | 8      | 0x00000002 |
| \$t1   | 9      | 0x00000000 |
| \$t2   | 10     | 0x00000000 |
| \$t3   | 11     | 0x00000000 |
| \$t4   | 12     | 0x00000000 |
| \$t5   | 13     | 0x00000000 |
| \$t6   | 14     | 0x00000000 |
| \$t7   | 15     | 0x00000000 |
| \$s0   | 16     | 0x00000004 |
| \$s1   | 17     | 0x00000004 |
| \$s2   | 18     | 0x00000000 |
| \$s3   | 19     | 0x00000000 |
| \$s4   | 20     | 0x00000000 |
| \$s5   | 21     | 0x00000000 |
| \$s6   | 22     | 0x00000000 |
| \$s7   | 23     | 0x00000000 |
| \$t8   | 24     | 0x00000000 |
| \$t9   | 25     | 0x00000000 |
| \$k0   | 26     | 0x00000000 |
| \$k1   | 27     | 0x00000000 |
| \$gp   | 28     | 0x10008000 |
| \$sp   | 29     | 0x7ffefd4  |
| \$fp   | 30     | 0x00000000 |
| \$ra   | 31     | 0x00400174 |
| pc     |        | 0x004001a4 |
| hi     |        | 0x00000000 |
| lo     |        | 0x00000000 |

Mars Messages

Run I/O

Go: execution paused at breakpoint: hofstadter.asm

Go: running hofstadter.asm

Go: execution paused at breakpoint: hofstadter.asm

Go: running hofstadter.asm

Go: execution paused at breakpoint: hofstadter.asm

Clear